- Memory Write Ability and Storage Permanence
- Common Memory Types
- Composing Memory
- Memory Hierarchy and Cache

1

4.1 Introduction

A memory stores large numbers of bits. For m words of each n bits, memory can store total of m*n bits. To access each word, address input signals are defined. $Log_2(m)$ address inputs are required to select m words. Also, if there are k address inputs then the memory can have 2^k words.

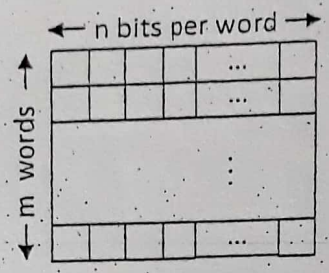


Figure 4.1: m x n memory

For example: A 4096 x 8 memory

- Stores 32768 bits
- Requires 12 (2^{12} = 4096) address signals
- Eight input/output data signals.

A memory access may refer to memory read – retrieve the word of a particular address, or memory write – store a word in a particular address. Control input signal r/w is used to indicate the type of access. Another control input signal, enable, which when asserted, is used to access the memory. Multiport memory supports multiple accesses to different locations simultaneously. Multiport memory systems have multiple sets of control lines, address lines, and data lines.

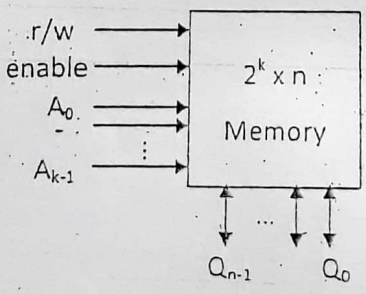


Figure 4.2: External View of Memory

Conventionally, ROM is referred as a memory that a processor can only read, and it holds stored bits even without a power source. Whereas, RAM is referred as a memory that a processor can both read and write but loses its stored bits if power is removed. But contemporarily, advanced ROMs, EEPROM and Flash, can be read as well as programmed and advanced RAMs, NVRAMs, can hold their bits even when power is removed. Advancement of memory have blurred the distinction

between the RAM and ROM. Different memories are differentiated based on two characteristics, write ability and storage permanence.

4.2 Memory Write Ability and Storage Permanence

Write Ability refers to the manner and speed that a particular memory can be written. Every memory must have a way to write bits onto it but the manner and speed of such writing varies among different memory. In-system programmable is used to categorize memories into two along the write ability axis. In-system programmable memory can be programmed by a processor whereas non in-system programmable memory must be programmed by some external means.

Range of Write Ability

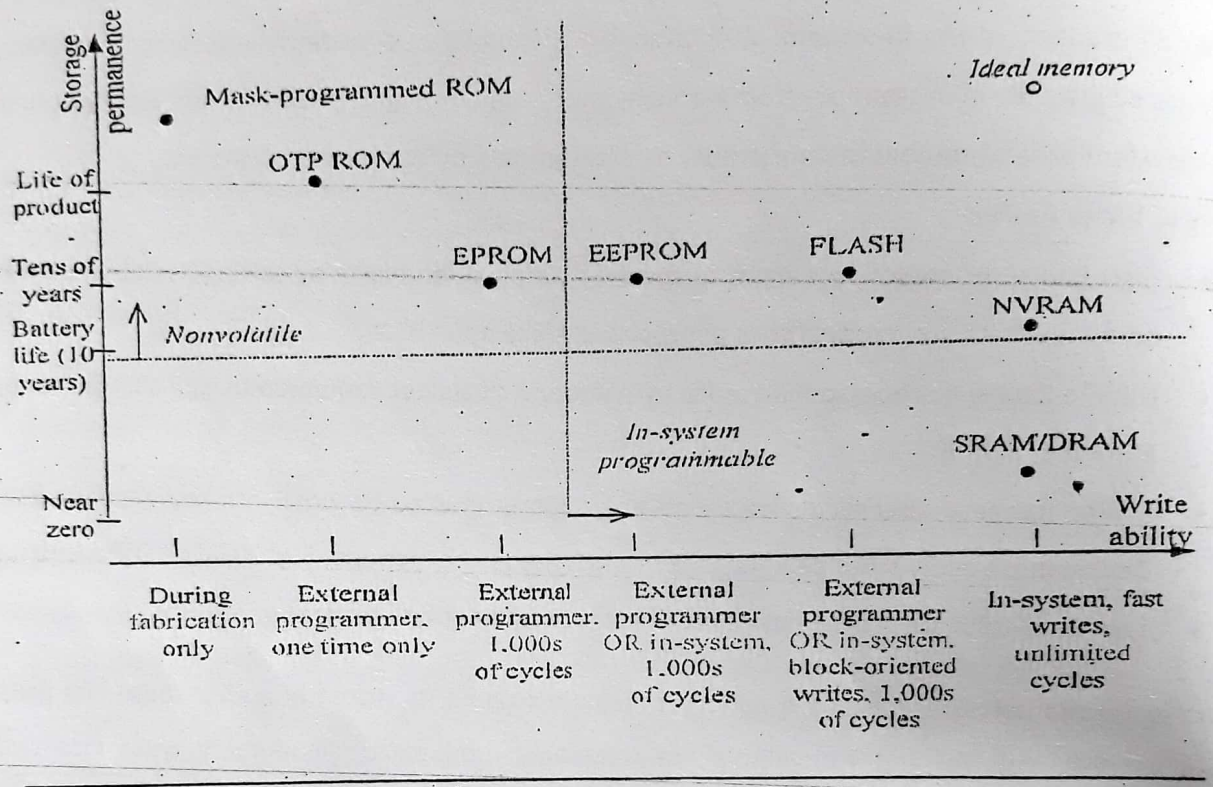
- High End – processor can write to memory simply and quickly by setting its address lines, data input bits and control lines appropriately. Example: RAM
- Middle Range – processor can write to memory a bit slower compared to high end. Example: EEPROM, FLASH
- Lower Range – special device called programmer is used to write into the memory. The device must apply suitable voltage levels to write to the memory. E.g.: EPROM, OTP ROM
- Low End – bits are stored during fabrication. Example: Mask-programmed ROM

Storage Permanence refers to the ability of memory to hold its stored bits after those bits have been written. Volatile and Nonvolatile are commonly used to divide memory types into two categories along the storage permanence axis. Non volatile memory can hold its bits even after power is no longer supplied. On the contrary, volatile memory requires continual power to retain its data.

Range of Storage Permanence

- Low End – memory in this range begins to lose its bits almost immediately after those bits are written and therefore it must be refreshed periodically. Example: DRAM
- Lower Range – memory holds bit as long as power is applied to the memory. Example: SRAM
- Middle Range – memory in this range holds bits for days, months, or even years after the memory power source has been turned off. Example: NVRAM
- High End – memory in this end will never lose its bits, as long as the memory chip is not damaged. Example: Mask Programmed ROM

Two characteristic, write ability and storage permanence, are important and desired in any system but it creates a trade-off. Write ability and storage permanence tend to be inversely proportional to one another. Moreover, highly writable memory, usually, requires more area and/or power than less-writable memory.



Write ability and storage permanence of memories, showing relative degrees along each axis (not to scale).

Figure 4.3: Various memories based on write ability and storage permanence

4.3 Common Memory Types

Read Only Memory (ROM): It is a nonvolatile memory, that can be read from but cannot be written to, by a processor, but it can be programmed by setting the bits within the memory. Traditionally, ROM is programmed off-line, when it is not actively involved within an embedded system.

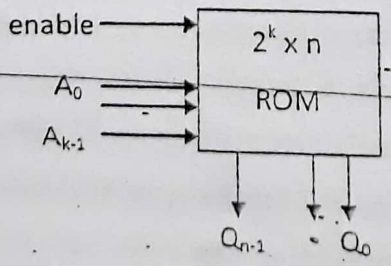


Figure 4.4: External Block Diagram

Uses of ROM

- store a software program for a general purpose processor
- To store constant data, like large lookup tables of strings or numbers
- to implement a combinational circuit

Example 1: Symbolic View of the internal design of an 8x4 ROM

- Horizontal lines = words (8), Vertical lines = data (4)
- Word line connected to data line via the programmable connections
- Circles on data and word lines are connected to represent high logic(1)
- Wired-OR represents all word lines are ORed together.
- If word 3 needs to be read then the input of decoder is set to 011 which makes the word 3 line high and other word lines low, since the data lines 0 and 3 are not connected to the high word 3 line, the output of the ROM will be 0110.

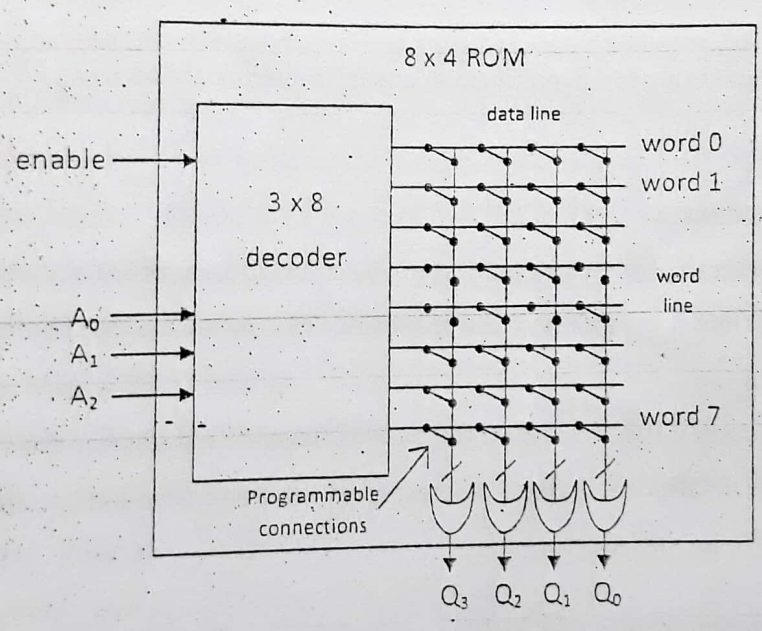


Figure 4.5: Internal View of an 8x4 ROM

Example 2: Implement the following combinational functions using a ROM

$y = a'b'c' + a'bc' + ab'c + abc$, $z = a'b'c + a'bc' + a'bc + ab'c + abc$

Solution: Three inputs a, b and c is taken as address lines. So, for three inputs the decoder of 3 x 8 must be used resulting in eight word lines. And there are two outputs, so there must be two data

lines. Hence, a ROM of 8 x 2 is required. Initially, the truth table, if not given, is formed from the given functions. The programming connections are done based on the output of the functions.

Inputs			Outputs	
a	b	c	y	Z
0	0	0	1	0
0	0	1	0	1
0	1	0	1	1
0	1	1	0	1
1	0	0	0	0
1	0	1	1	1
1	1	0	0	0
1	1	1	1	1

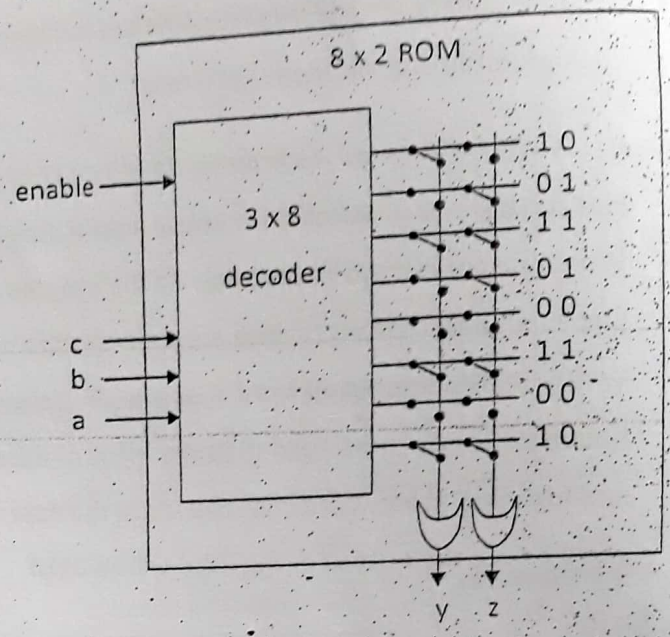


Figure 4.6: Truth table for given function and its implementation using ROM

TYPES OF ROM

A. Mask-Programmed ROM

- Connection is programmed during fabrication, by creating an appropriate set of masks.
- It has extremely low write ability. Once fabricated, its content cannot be reprogrammed or changed.
- It has highest storage permanence. Stored bits will never change unless the chip is damaged.
- It is used in such embedded systems whose design has been finalized and large numbers of unit are needed to be manufactured.

B. One-Time Programmable ROM – OTP ROM

- Connection is programmed using a device called programmer that configures each programmable connection according to the file provided by user. Programmer blows fuses by passing a large current wherever a connection is not required. The blown out fuses cannot be reestablished, hence it is referred as one time programmable ROM.
- It has lowest write ability of all PROMs, since it can be programmed only once.

- It has very high storage permanence, since its stored bits won't change unless some more fuses are blown out using programmer.
- It is cheap which makes it more suitable in final products compared to other types of PROM. Also compared to mask-programmed ROM, time-to-market constraints and unit costs make OTP-ROM a better choice.

C. Erasable Programmable ROM – EPROM

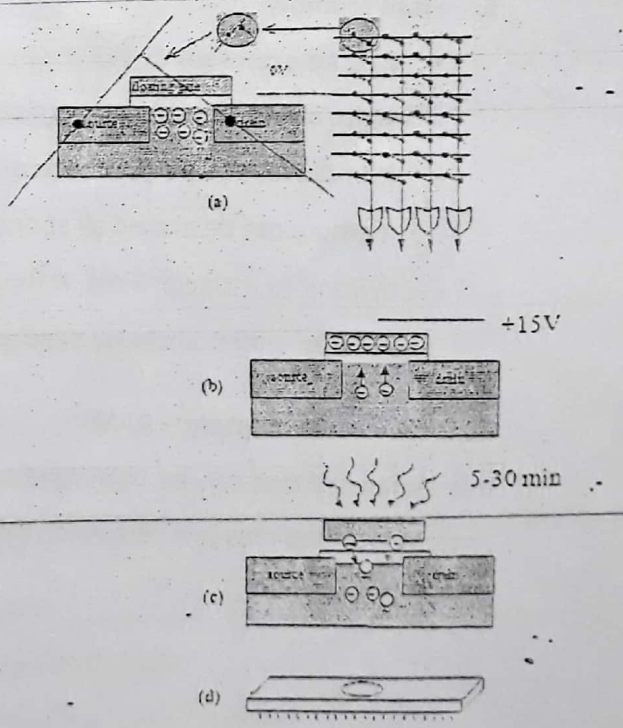
- EPROM uses a MOS transistor as its programmable component. The transistor has a floating gate surrounded by insulator. When high voltage (12v – 25v) is applied, it causes electrons to tunnel through the insulator into the gate. When the high voltage is removed, the electrons cannot escape and hence the gate has been charged and programming has occurred. To erase the program, the electrons must be excited enough to escape the gate which is done by exposing UV light for 5 – 30 minute. For the UV light to reach the chip, EPROMs are provided with a small quartz window in the package.
- Reading an EPROM is much faster than writing, since reading doesn't require programming.
- EPROMs have improved write ability and can be reprogrammed thousands of times.
- EPROMs have reduced storage permanence. They hold their stored bits for about 10 years.
- Electrical noise or radiations causes stored bits of the chip subject to undesirable changes and hence EPROMs are scarcely used in production. It offers a better choice in the testing phase of the system rather than in production.
- Internal Operation of EPROM

a) Negative charges form a channel between source and drain storing a logic 1

b) Large positive voltage at gate causes negative charges to move out of channel and get trapped in floating gate storing a logic 0

c) Shining UV rays on surface of floating gate causes negative charges to return to channel from floating gate restoring the logic 1

d) An EPROM package showing quartz window through which UV light can pass



D. Electrically Erasable Programmable ROM – EEPROM

- EEPROM is programmed and erased electronically, using higher than normal voltage. Electronic erasing requires seconds, rather than many minutes required for EPROMs. Moreover, individual words can be erased and reprogrammed in case of EEPROM, whereas EPROM can only be erased in their entirety.
- It is in-system programmable since circuit providing higher than normal voltage levels for erasing and programming is built into the embedded system. EEPROM is built with a built in memory controller which hides internal memory access details for the memory user and provides a simple memory interface to the user. The memory controller contains the circuitry and single purpose processor required to erase and program the word at the user specified address.
- EEPROM provides better write ability compared to EPROM, it can be reprogrammed tens of thousands of times.
- EEPROM has storage permanence on a par with EPROM, about 10 years.
- Writing is slower, since it involves the process of erasing and programming. Busy pin is available to indicate that the EEPROM is busy in writing.
- EEPROM can be used to serve as the program memory for a microprocessor. It can also be used to store data than an embedded system should save after the system is off.

E. Flash Memory

- It is an extension of EEPROM which uses the same floating-gate principle along with same write ability and storage permanence.
- It improves the performance of a system with its fast-erase ability, in which large blocks of memory can be erased all at once.
- Writing to a single word in flash may be slower than writing to a single word in EEPROM, since an entire block will need to be read, updated and written back.

Random Access Memory – RAM

It is a memory that can be both read and written easily. Typically RAM is volatile, since it loses its content after the power is removed. The internal structure of RAM is comparatively complex than of ROMs.

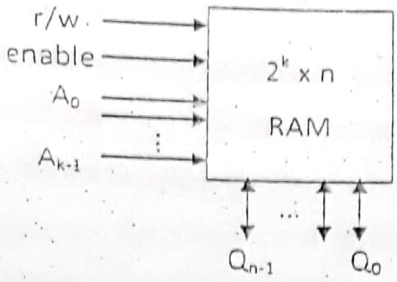


Figure 4.7: External View of RAM

Example 1: Sketch the internal structure of a 6 x 6 RAM

- Each word consists of a number of memory cells, each storing one bit.
- Each input data line and output data line is connected to every cell in its column.
- Output of a memory cell being ORed with the output data line of each column.
- The read/write input is connected to every cell

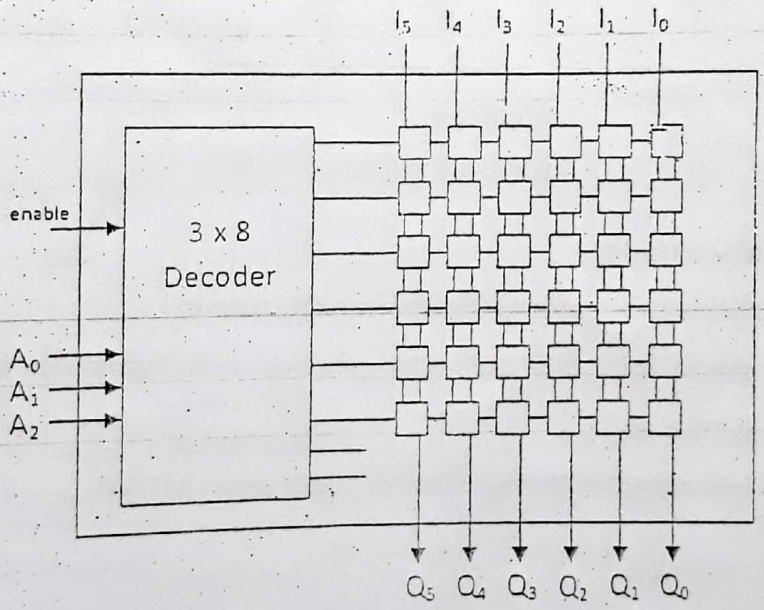


Figure 4.8: Internal Structure of 6 x 6 RAM

Types of RAM

A. Static RAM – SRAM

- Uses a memory cell consisting of a flip flop to store a bit
- Requires about six transistors to represent a single bit
- It holds data as long as power is supplied hence called static RAM.
- Generally used for high-performance parts of a system. E.g. Cache memory

B. Dynamic RAM

- Uses a memory cell consisting of a MOS transistor and capacitor to store a bit
- Requires only one transistor, resulting in more compact memory than SRAM
- Each cell must be charged (refreshed) regularly, since the charge stored in capacitor leaks gradually causing the loss of data.
- DRAM access tends to be slower than SRAM, since accessing a DRAM word results in the word's being stored in a buffer and then being written back to the word's cell.

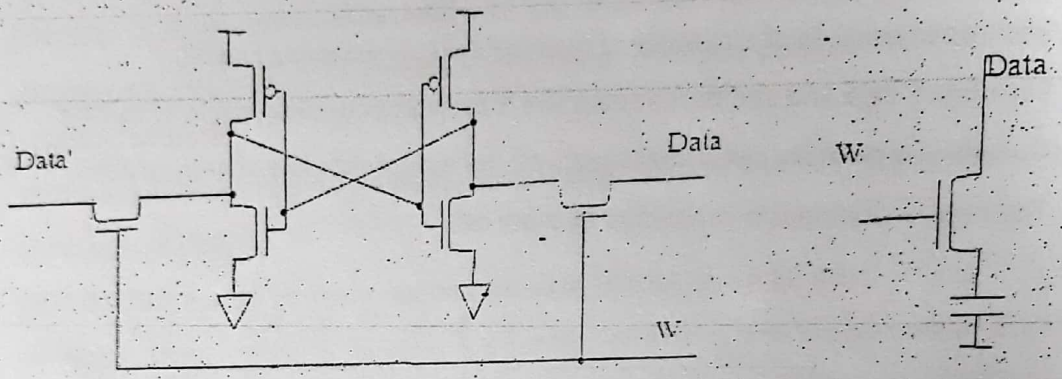


Figure 4.9: SRAM and DRAM

C. Pseudo-Static RAM – PSRAM

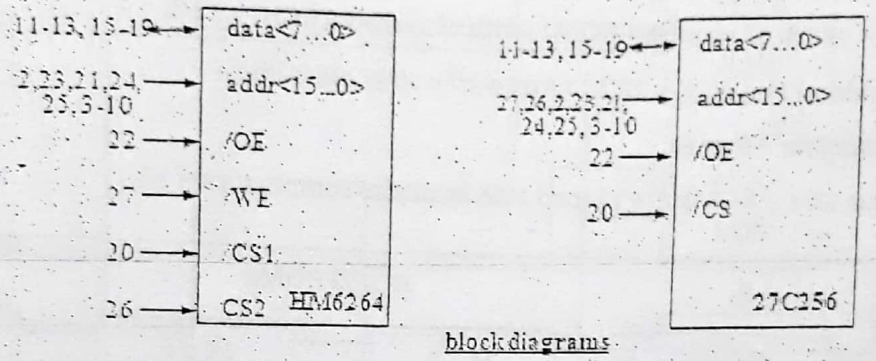
- These are DRAMs with a memory refresh controller built in.
- PSRAM may be busy refreshing itself when accessed, which could slow access time and add some system complexity.
- It is a popular low-cost high-density memory alternative to SRAM.

D. Nonvolatile RAM – NVRAM

- It holds data even after external power is removed.
  - Battery-Backed RAM: Contains a static RAM with permanent battery connected. When power is removed or drops below a certain threshold, the internal battery maintains power and the memory continues to store its bits. There is no limit on the number of times the Battery-Backed RAM can be written to.
  - Static RAM with EEPROM or FLASH: This type of NVRAM stores its complete RAM contents into the EEPROM just before the power is turned off. The data is reloaded into RAM after the power is turned back in.

Example: HM6264 and 27C256 RAM/ROM Devices

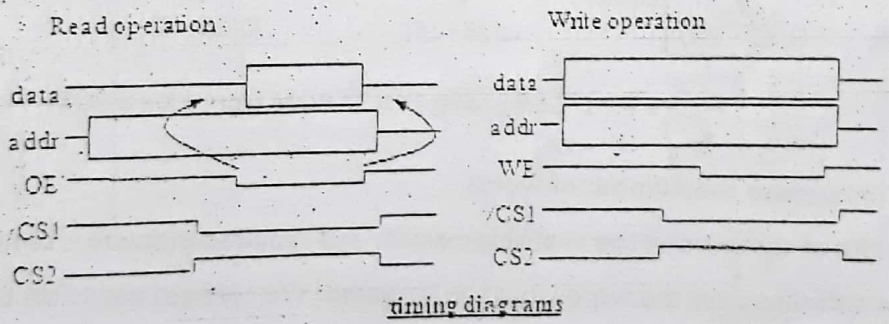
- Low-cost low-capacity memory devices used in 8-bit microcontroller-based embedded systems
- The first two numeric digits indicate whether the device is RAM (62) or ROM (27), whereas the subsequent digits give the memory capacity in kilobits.
- Placing a memory address on the address-bus and asserting the read signal output enable (OE) performs a read operation.
- Placing some data and a memory address on the data and address busses and asserting the write signal enable (WE) performs a write operation.



block diagrams

Device	Access Time (ns)	Standby Pwr. (mW)	Active Pwr. (mW)	Vcc Voltage (V)
HM6264	85-100	.01	15	5
27C256	90	.5	100	5

device characteristics



timing diagrams

Figure 4.10: Example - HM6264 and 27C256 RAM/ROM Devices

4.4 Composing Memory

Composing Memory is needed when there is a need of particular-sized memory, which is not readily available. If the available memory is larger than required one, then we simply use the needed lower words of the memory and ignore the higher words which are not required. However, if the available

memory is smaller than needed, some more design procedures are needed to be followed. The various cases for composing memory have been discussed in the following paragraphs.

A. Case 1: To increase the width of words

When the number of words in the available memory is same to that of required one but the number of bits or width of word is not enough then the width must be increased. To do that, the available memories are connected side by side as shown in the given example.

Example 1: Compose 1K x 8 ROMs into a 1K x 32 ROM

Analysis: The available ROM 1K x 8 and required ROM of 1K x 32 have same number of words but width is different. The number of ROM to be placed side by side is given by n.

- n = width of required ROM/width of available ROM = 32/8 = 4
- Address line = 1K = 1024bytes = 2^{10} = 10 address lines.
- Data line = 8 lines

Hence, four 1K x 8 ROMs are placed side by side to compose 1K x 32.

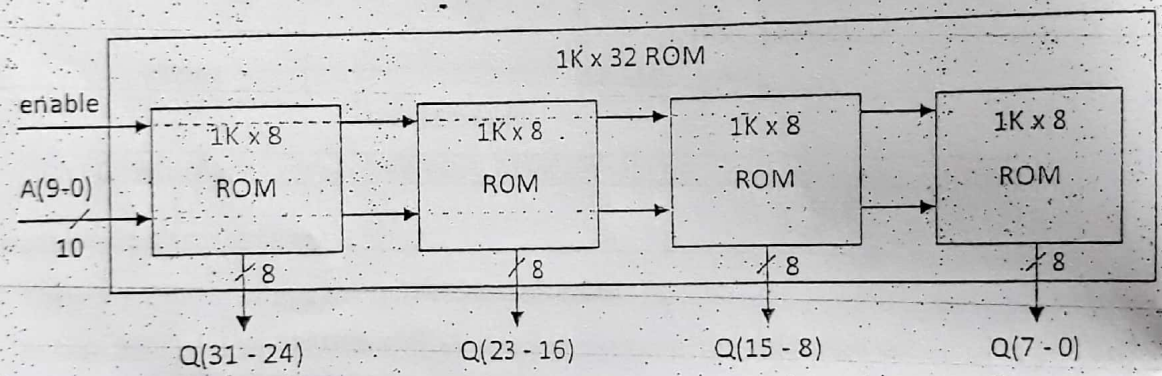


Figure 4.11: Composing 1K x 32 ROM from 1K x 8 ROM

B. Case 2: To increase the number of words

When the width of the word in the available memory and required memory is same but the number of words are different then the words must be increased. We connect the ROMs top to bottom and data line of each ROM is ORed. Since the number of words has to be increased, extra high-order address is required to select the particular ROM which can be implemented by using appropriate decoder.

Example 1: Compose 1K x 8 ROMs into a 4K x 8 ROM.

Analysis: The available ROM 1K x 8 and required ROM 4K x 8 have same width of 8 bits but the number of words is different. Number of ROMs and the size of decoder can be determined as

- N = number of words in required ROM/number of words in available ROM = 4K/1K = 4
- Decoder: It must be able to select 4 ROM, so 2 x 4 decoder must be used.
- Higher address bits = $\log_2(4K) - \log_2(1K) = \log_2(2^{12}) - \log_2(2^{10}) = 12 - 10 = 2$ bits or lines
- Total Address line: $4K = 2^{12} = 12$ address lines, and 10 lines (A_9 to A_0) are connected to each ROM. 2 higher address is represented by inputs of decoder.

Hence, four ROM must be connected top to bottom and data line of each ROM is ORed. Decoder of 2 x 4 is used to select a particular ROM.

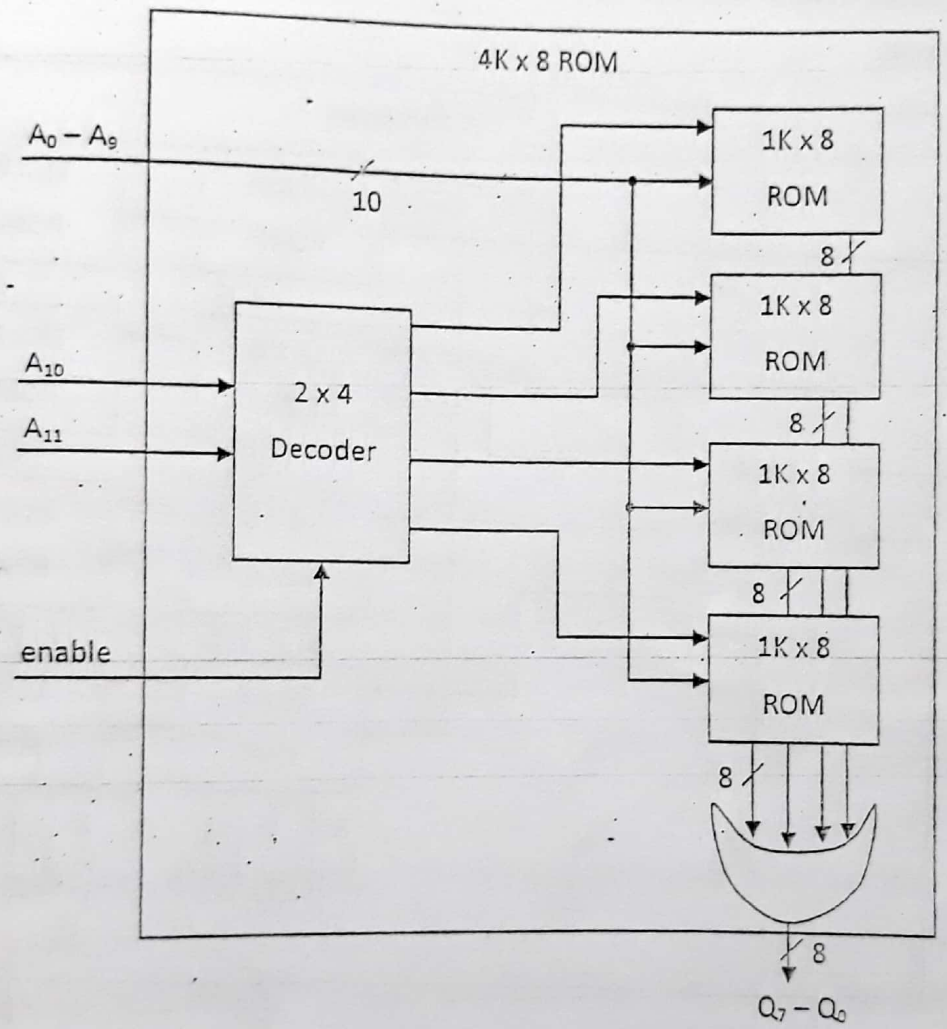


Figure 4.12: Composing 4K x 8 ROM using 1K x 8 ROM

C. Case 3: To increase both, number of words and word width

When the width of the word as well as the number of words in the available memory and required memory are different then the technique used in case 1 and case 2 must be combined. Initially, the number of words is increased and then the top-bottom set of ROMs with ORed data lines are placed side by side to increase the word width.

Example 1: Compose 1K x 8 ROMs into a 4K x 16 ROM

Analysis: The available ROM 1K x 8 and required ROM 4K x 16 differ in number of words as well as word width.

- Increase number of words: 4k/1k = 4, Four ROMs are required with 2 x 4 decoder. 4K represents 12 address lines, 10 lines connected to every ROM and 2 lines represented by inputs of decoder.
- Increase word width: 16/8 = 2, Four set of ROMs are repeated two times and placed side by side.

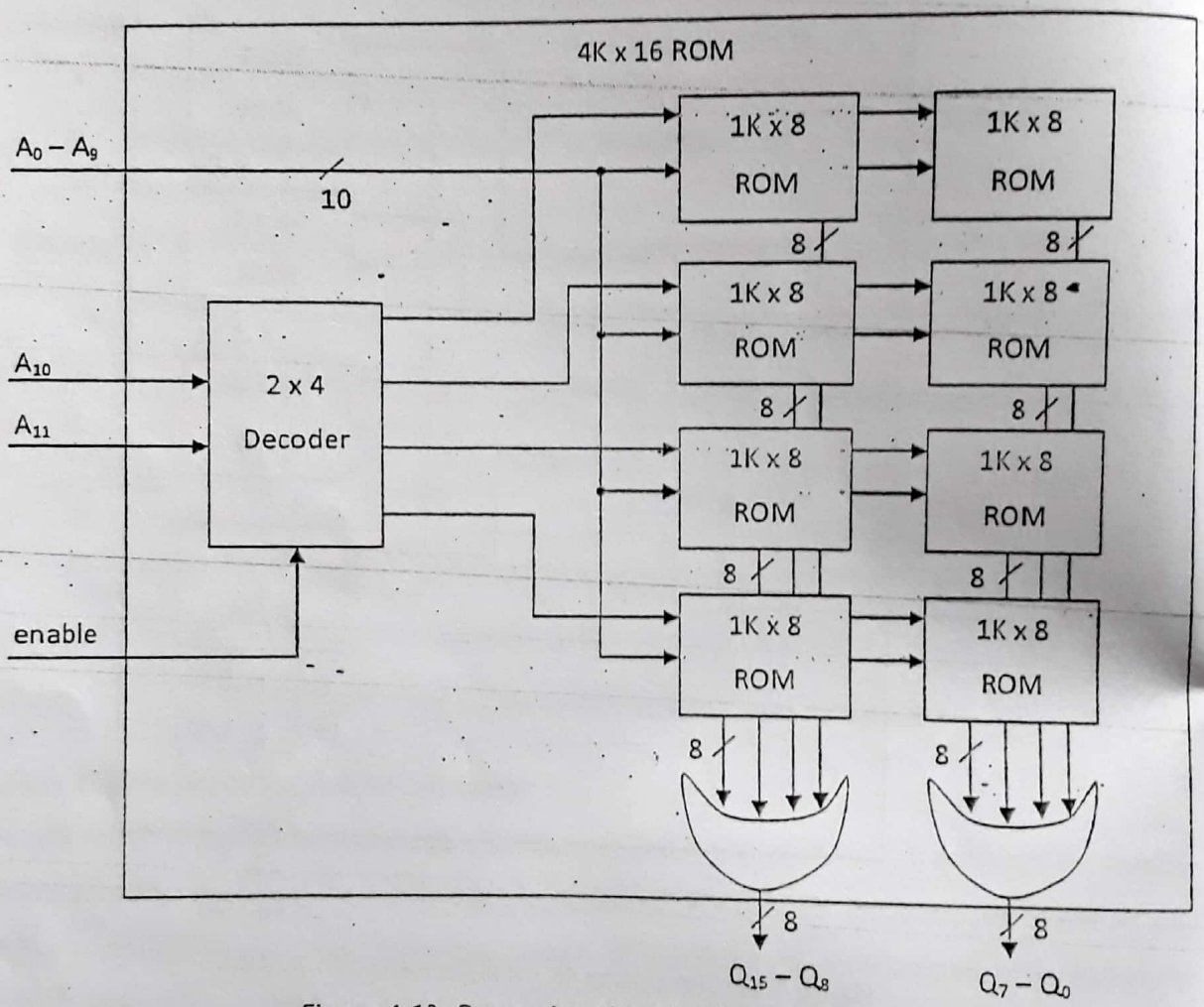


Figure 4.13: Composing 4K x 16 using 1K x 8 ROM

4.5 Memory Hierarchy and Cache

Memory Hierarchy

A system cannot be implemented with only fast memory as it makes the system very expensive. Also the use of only slow and low cost memory will make system very inefficient. So, the concept of

memory hierarchy comes into action in which a system is more likely to implement slow but high capacity memory for storage along with fast but small memory for high speed processing. Memory hierarchy defines the level of memory based on cost per bit, capacity and access time. As we move down the hierarchy, capacity increases, access time increases and cost per bit decreases.

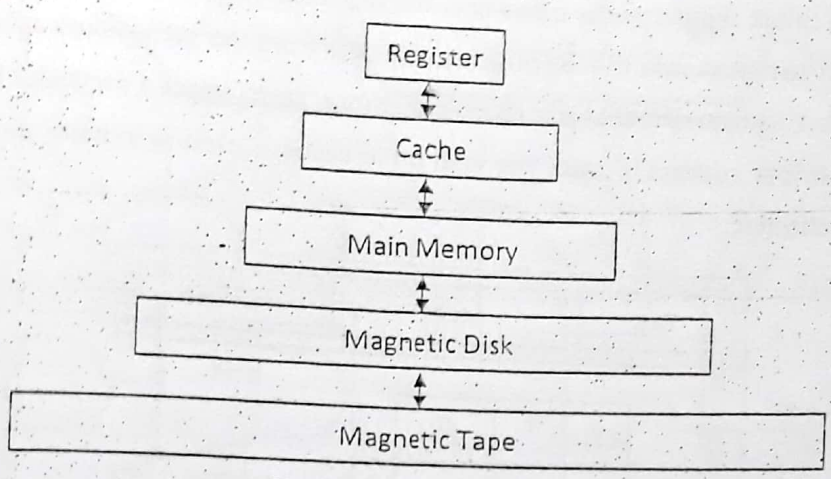


Figure 4.14: Memory Hierarchy

Cache Memory

Cache is a small but fast memory which contains a copy of portions of main memory to expedite operations of the system. Cache is designed using static RAM which makes it faster as compared to main memory. The access time for cache can get as low as one clock cycle while main memory access requires several cycles. So, the instructions and data which are supposed to get accessed frequently are placed in cache memory. Hence, the average access time is reduces resulting in improved performance.

During cache operation, the processor first checks the required word in cache. If it is available (cache hit), the word is delivered to the processor. But, however, if the word is not available (cache miss) in cache then the corresponding block of main memory is read into cache. And finally the word is made available to the processor. This operation leads to various cache design issues which are discussed in the following paragraph.

A. Cache Mapping Techniques

Cache memory is very small as compared to main memory. And all blocks of main memory cannot be assigned to cache memory at once. So, cache mapping techniques are required to assign particular block of main memory to the appropriate line in cache memory. There are basically three types of mapping techniques which are discussed below.

Direct Mapping

In this technique, main memory block is assigned to a fixed cache line. The cache stores the content of main memory, the tag and the valid bit. Here, the memory address is divided into the tag, the index and the offset. The index, which is defined by the cache size, represents the cache address. Index is used to select the particular cache line. The tag from main memory address is compared to the tag stored in cache. In case the tag matches, the data from the cache line is accessed. However, a single cache line can store few blocks of main memory. So to select a particular block, the offset part of main memory address is used. The valid bit in cache is used to indicate the validity of data stored in the cache slot.

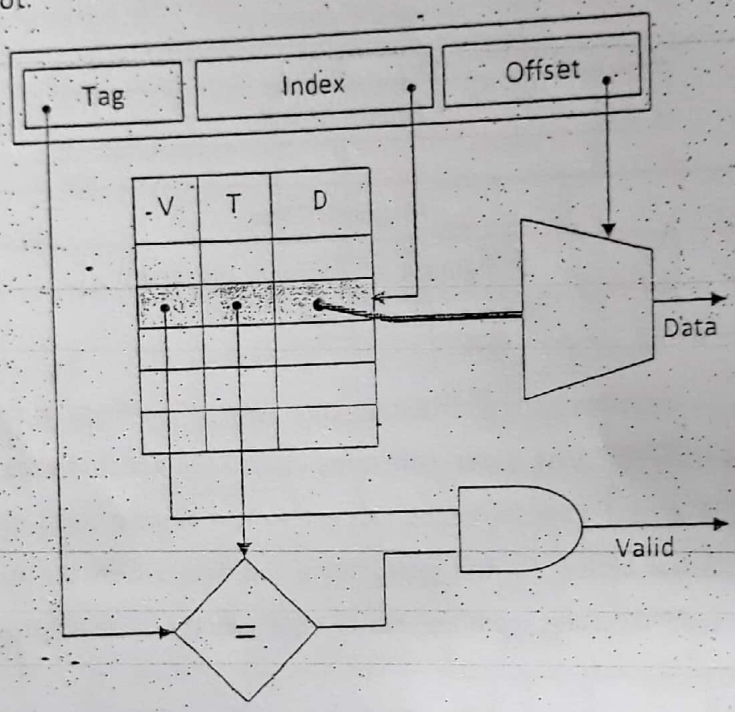


Figure 4.15: Direct cache mapping

Direct cache mapping is easy and simple in implementation. However, when two blocks of main memory which are assigned to a particular cache line are to be accessed frequently, then cache miss occurs repeatedly. This problem is commonly referred as thrashing. Also, replacement algorithm cannot be used, since main memory blocks are mapped to a fixed cache line.

Fully Associative mapping

In this mapping, main memory block can be assigned to any slot of cache line. The main memory address is divided into tag field and offset field. The tag from main memory is compared to each tag in the cache line. After the tag matches the offset is used to select a particular word in cache line.

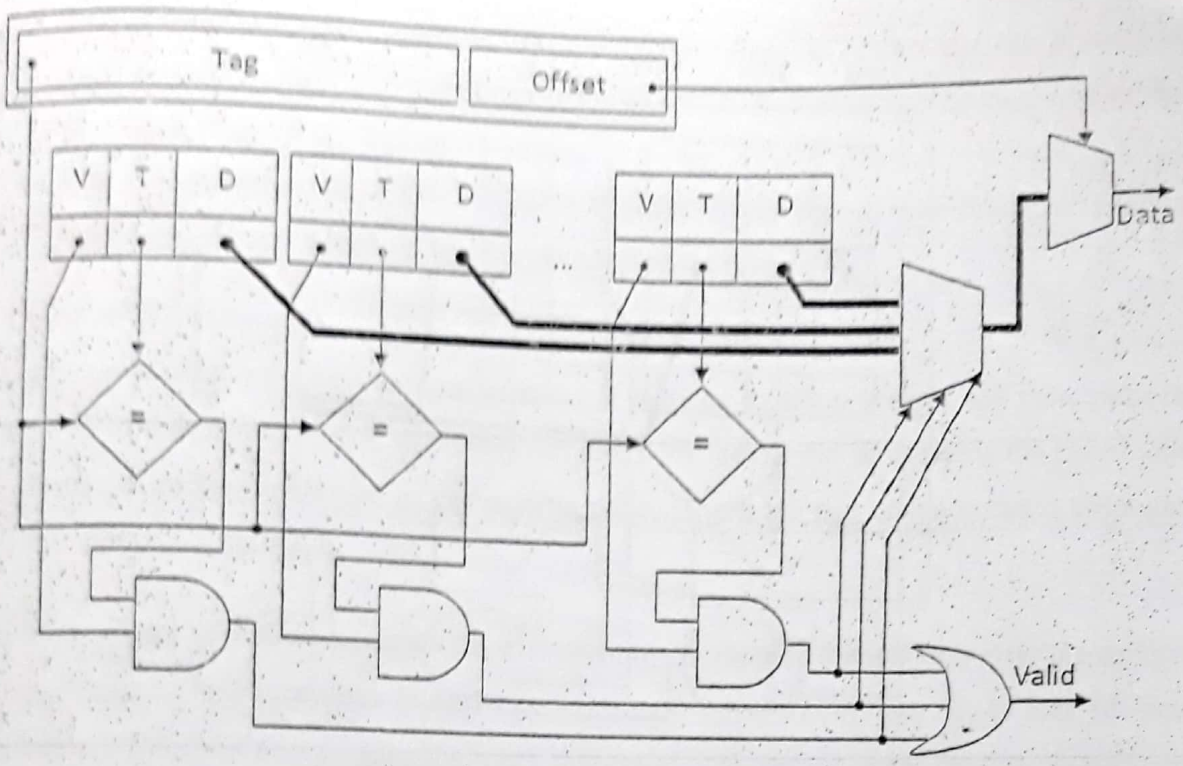


Figure 4.16: Fully associative cache mapping

Fully associative mapping provides high flexibility as block of main memory can be assigned to any cache line. However, the comparison logic is required for each cache line which makes this mapping method complex and expensive to implement. Miss rate can increase if frequently required block is replaced, so appropriate replacement algorithm must be utilized for efficient cache implementation.

Set Associative Mapping

It is a compromise mapping which, somehow, follows both direct and fully associative mapping. The cache is divided into sets, each with number of cache lines. A cache with a set of size N is called an N-way set associative cache. Each block of main memory can be mapped to particular line of any sets (fixed line but varying sets) or any lines of particular set (fixed set but varying lines). Taking former case into consideration, the main memory address is divided into tag, index and offset. The index field is used to select the fixed cache line, and the tag field of main memory is compared to tag of each sets. When the particular set is selected, the offset is used to select the particular word from the set in which the tag matches.

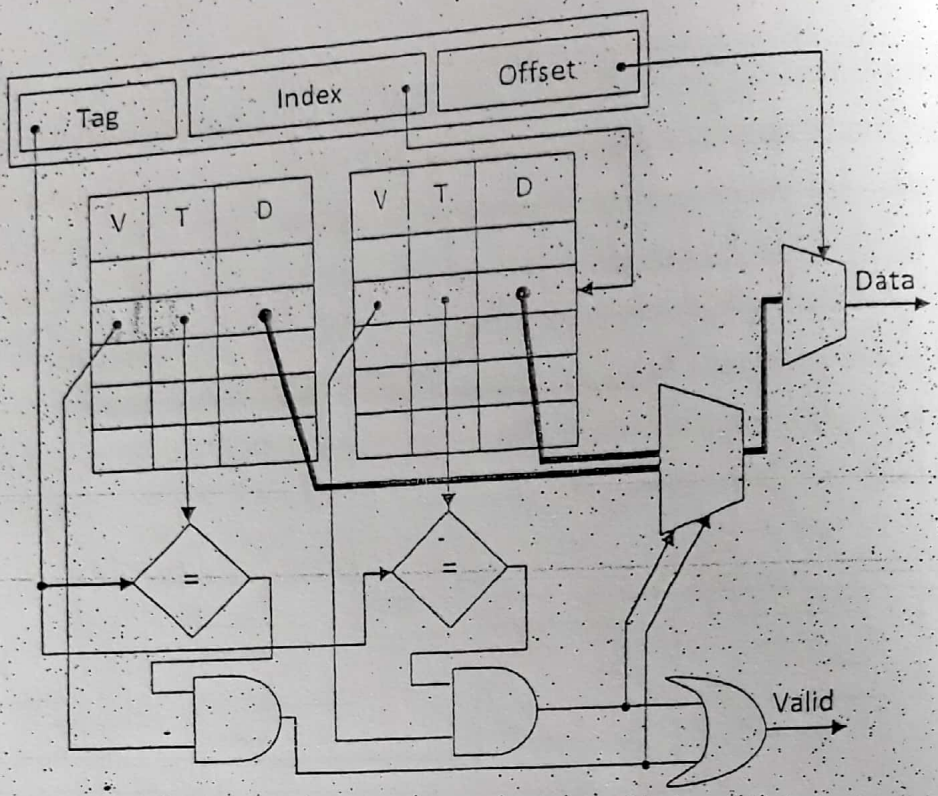


Figure 4.17: Two-way set associative

Set associative cache mapping is more flexible and can reduce cache misses as compared to direct mapping. Though the block of main memory is assigned to fixed cache line, block can be assigned to any sets of cache line. And proper implementation of cache replacement can be used to increase cache hit rate. Also the comparison logic is not required for every cache line rather is required for only available sets which reduce the complexity and expense for implementing comparison logic.

B. Cache-Replacement Policy

When cache is full and new main memory block is to be assigned to the cache then certain technique must be used to choose which cache line should be replaced. This mechanism of replacing the existing block by new set of blocks is referred as cache-replacement policy. In direct mapping the main memory block always maps to the fixed cache line, so replacement is fixed. But fully associative and set associative can follow various replacement algorithms. Least Recently Used (LRU), First In First Out (FIFO), Least-Frequently Used (LFU) and Random are few commonly used replacement techniques.

- Random replacement replaces the block randomly without following any specific algorithm.
- Least Recently Used (LRU) algorithm is based on time in which the block not accessed for longest time is replaced by the new block.

- First In First Out (FIFO) method uses queue mechanism to replace the first entered block. Each block is pushed into the queue when accessed. And when replacement is required the blocks are popped out from the queue.
- Least Frequently Used (LFU) technique is based on number of time the block is accessed. The block which is accessed less number of times is replaced.

C. Cache Write Techniques

A mechanism is required when content of cache is changed by the processor and the change must be updated to the corresponding main memory block. This technique of updating the main memory after change in cache is referred as cache write policy. There are two common cache write policy; write-through and write-back.

Write-through is a technique in which the main memory is updated immediately after the content in cache is changed. This technique is easier to implement but the processor has to wait for slower main memory frequent access. Also there are chances of unnecessary writes resulting in substantial memory traffic. For example when a particular value is changed four times, the last updated value must only be updated in the main memory. But the memory is updated four times for every change causing unnecessary memory access.

Write-back policy allows main memory to be updated only when cache line is to be replaced. Extra bit is associated with each cache line to represent whether the content of cache line is changed or not. Based on that extra bit the corresponding main memory block is updated when cache line is about to be replaced. Extra bit and update checking increase system complexity; however, it reduces number of slow main memory access and avoids memory congestion.

D. Cache Impact on System Performance

The performance of system is directly related to design and configuration of caches. The total size of cache, degree of associativity, and the data block size are important parameters that have direct impact on performance.

Cache size is the total number of bytes that the cache can store. The tags and extra bits, which do not contribute to the size of the cache, are also stored in cache along with the data of main memory block. Increasing the size of cache results in lower miss rates, however the access of data from the cache will be slower. So, larger cache size does not necessarily mean better performance.

Degree of associativity is related to number of sets used in set associative cache implementation. Increasing the number of sets will improve the hit rate. However, additional logic requirement will increase the access time latency.

Cache line size represents the size of each block in cache that holds the block of data of main memory. When line size is increased, the main memory access time is, obviously, reduced but only at the expense of more complex multiplexing circuitry which increases the access latency.

Example: Effect of cache size on system performance

Case I: Cache size = 2Kbytes, miss rate = 15%, hit cost = 2 cycles, miss cost = 20 cycles

Average cost of memory access = (0.85 x 2) + (0.15 x 20) = 4.7 cycles

Case II: Cache size = 4Kbytes, miss rate = 6.5%, hit cost = 3 cycles, miss cost = 20 cycles

Average cost of memory access = (0.935 x 3) + (0.65 x 20) = 4.105 cycles

Case III: Cache size = 8Kbytes, miss rate = 5.565%, hit cost = 5 cycles, miss cost = 20 cycles

Average cost of memory access = (0.94435 x 2) + (0.05565 x 20) = 4.8904 cycles

In case II, increase in cache size, certainly, improved the performance as average cost of memory access is decreased. However, in case III, increase in cache size added more cycles for memory access in average.

Advanced RAM

A. Fast Page Mode Dram (FPM DRAM)

FPM DRAM is asynchronously controlled which is designed with some improvements on the basic DRAM architecture. In this design, each row of the memory bit-array is viewed as a page which contains multiple words. Each word is addressed by a unique column address. In its operation, first the row or page address is sent and then the corresponding column address must be sent to read a particular word. In each memory cycle, three data words can be read consecutively by providing their corresponding column address. Hence, it eliminates the requirement of extra cycle as three cycles would have been required to read three words.

B. Extended Data Out DRAM (EDO DRAM)

EDO DRAM is similar to FPM DRAM with additional feature that reduces the read/write latency. Here, new access cycle can be started while keeping the data output of previous cycle active. In simple words, new column address can be sent while reading previously selected word from the

memory. This results in overlapping of the operation which reduces the latency of memory access. However, extra output latch must be introduced in the architecture.

C. Synchronous DRAM (SDRAM)

In SDRAM, the information is latched to and from the controller on the active edge of the clock signal. The time required to detect the strobe signals in asynchronous DRAM is eliminated by SDRAM. This DRAM architecture can have additional column address counter which holds the starting address of the data to be accessed. This counter is incremented internally to provide new data in each clock cycle as long as the data required are consecutive memory locations. The enhanced synchronous DRAM (ESDRAM), is the improved version of the SDRAM. ESDRAM provides faster clocking and lower latency in reading and writing data.

D. Rambus DRAM (RDRAM)

Rambus represents the bus interface architecture which uses multiplexed address/data lines to connect the processor to the RDRAM device. RDRAM may be further divided into number of banks with each remain open for access. Multiple open page scheme and fast bus I/O can result in high throughput. However, as compared to other standards, Rambus showed increase in latency, heat output, complexity, and cost. Requirement of heat-spreaders along with packet demultiplexors makes it more complex while manufacturing. More complex interface circuitry and more number of memory banks increased the size and resulted to become expensive.

E. Double Data Rate SDRAM (DDR SDRAM)

The DDR SDRAM is capable of making higher transfer rates with more strict control of the timing of the data and clock signals. The interface transfers data on both the rising and falling edges of the clock signal to double the data bus bandwidth. DDR SDRAM also known as DDR1 was replaced by DDR2 which operated on same principle but for higher clock frequency and produced double throughput as compared to DDR1. Similarly, DDR3 and DDR4 offered better performance for increased bus speed and new features.

Memory Management Unit (MMU)

Memory Management Unit is a processor which translates the logical address to physical memory address. MMU has important role in handling DRAM refresh, bus interface and arbitration used in memory. In addition, it takes care of memory sharing among multiple processors. Contemporary CPUs have built-in MMU as a part of processor.